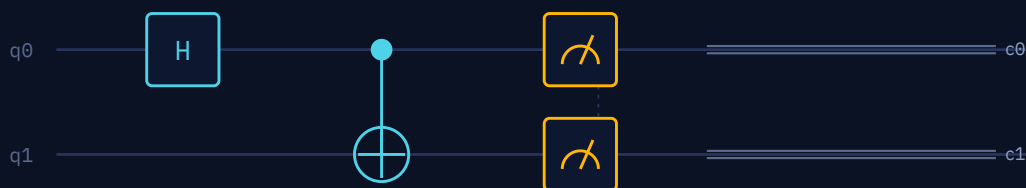


A step-by-step build guide

From account to your first quantum circuit

Setting up an IBM Quantum account, reading the Composer's block-diagram editor, configuring Linux for Qiskit, and running a real circuit on IBM's hardware.



GUIDE MAP

What's in this guide

Part	Section	Page
01	Create your IBM Quantum account, API key and instance	03
02	Reading the Composer — the block-diagram circuit editor	04
03	Build and run a Bell state in the Composer	05
04	Set up Linux for Qiskit — your package manager, Python, a virtual environment	06
05	Save your IBM credentials for Python	07
06	Your first Qiskit program — build and simulate locally	08
07	Run on real IBM hardware, and on a noisy simulator	09
08	How the Composer and Qiskit code connect	10
09	Reference — bit order, gotchas, and a one-page cheat sheet	11



Before you start

You'll need

- A Linux PC — Ubuntu/Debian, Fedora, or Arch (steps cover all three)
- An internet connection and about 30–45 minutes
- An email address, for the free IBM Quantum Open Plan
- A terminal (try Ctrl+Alt+T), used for Part 04 onward

What you'll build

- A Bell state — two entangled qubits — first in the visual Composer, then in Python
- A working local Qiskit environment on your machine
- One circuit run for real on an IBM quantum processor

A NOTE ON COST

Everything here uses IBM's free **Open Plan** — a rolling monthly allowance of processor time, far more than this guide needs. Composer's visualizer and the local simulator use none of it at all, so it's worth practising there first.

01

Create your IBM Quantum account

Composer and the Python SDK both run on top of one IBM Quantum Platform account. Set it up once here, and you can use either interface for the rest of this guide.

01 Open the platform and sign in.

Go to `quantum.cloud.ibm.com` in Firefox or Chrome, and log in or create an account. This is the same account used by both Composer and the API.

02 Land on the dashboard.

You'll see your available compute time, your instances, and recent jobs. The default free instance is created for you automatically on the Open Plan.

03 Generate an API key.

From the dashboard, create an API key and copy it somewhere safe immediately — it is shown only once. This key is what your machine uses to authenticate later, in Part 05.

04 Copy your instance CRN.

Open the *Instances* page, hover over your instance's CRN (a long resource identifier), and click to copy it. You'll pair it with the API key.



FIG 01 — Where the API key and instance CRN come from, and where they're headed next.

OPEN PLAN LIMITS

The free Open Plan currently provides up to **10 minutes of QPU time per 28-day rolling window**, and free instances can only be created in the **us-east** region. Active Open Plan users can additionally opt in to an extra 180 minutes across 12 months. A small two-qubit circuit like the one in this guide typically uses only a few seconds.

02

Reading the Composer

This is "the block-diagram thing" — IBM's drag-and-drop circuit editor. It's four coordinated panels around one circuit. Open it from the **Composer** link in the platform's top menu.

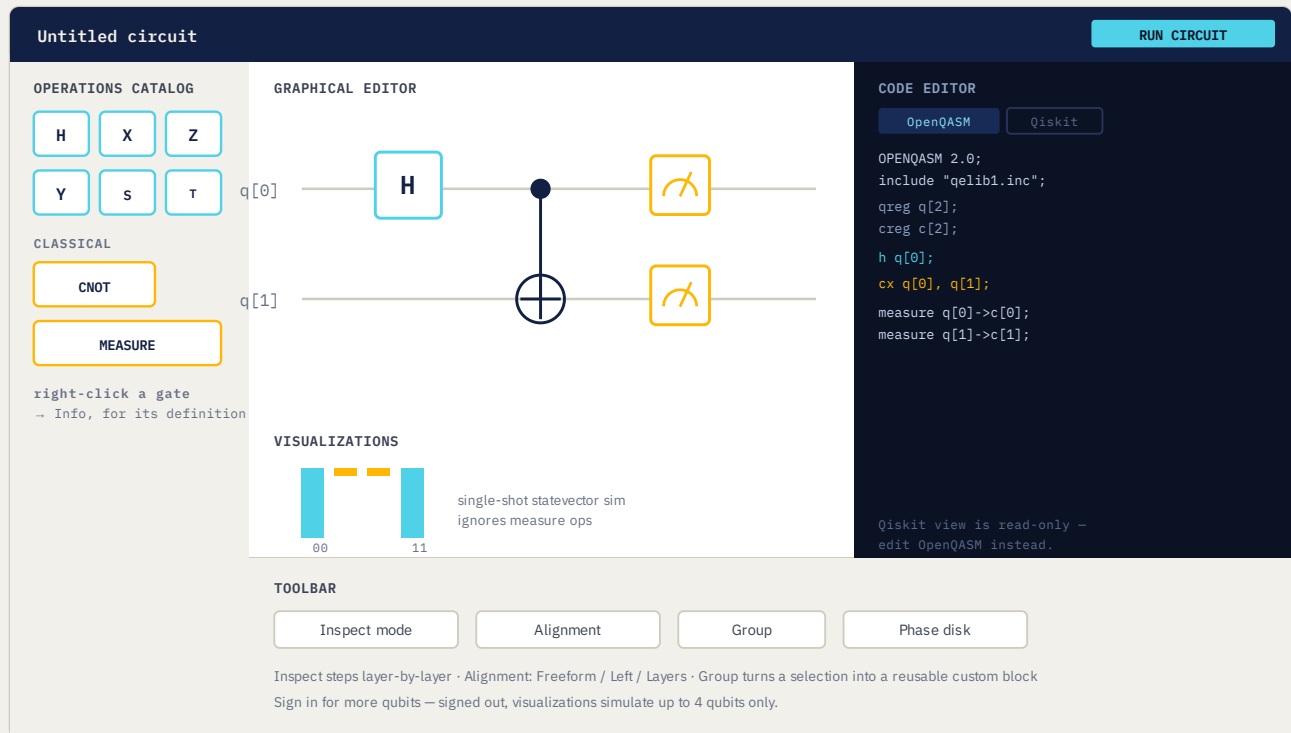


FIG 02 — The Composer's four panels: operations catalog, graphical editor, code editor, and visualizations — shown here mid-way through building the Bell state from Part 03.

03

Build a Bell state in the Composer

A Bell state entangles two qubits so their measurement outcomes always match. It's the simplest circuit that shows quantum behaviour a classical circuit can't reproduce.

01 Name your circuit.

Click "Untitled circuit" at the top and give it a name.

02 Add a Hadamard.

Drag an **H** gate onto $q[0]$. This puts it into superposition.

03 Add a CNOT.

Drag **CNOT** from the classical gates group onto the wires, with control on $q[0]$ and target on $q[1]$. If it's flipped, select it and click *Edit*.

04 Add measurements.

Drag **MEASURE** onto both qubits.

05 Check the result.

The probabilities view should show ~50% for 00, ~50% for 11, and 0% for 01 and 10.

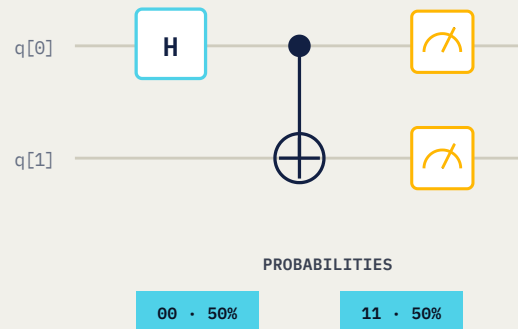


FIG 03 – The finished Bell-state circuit and its expected probability split.

INSPECT MODE

Toggle it in the toolbar to step through the circuit layer by layer — useful for seeing the state change one gate at a time. You can't add gates while it's on.

Running it on real hardware, from here

01 Click Run circuit,

top right.

02 Pick a QPU, your instance, and a shot count.

Leave the QPU on default to auto-select the least busy one.

03 Click Run, then View jobs

to open the Workloads page. A finished job shows a histogram plus the original and transpiled circuits — expect a few stray 01/10 counts from real device noise.

SAVE YOUR WORK

The Composer session isn't kept between visits — use **File** → **Save file** (OpenQASM 2.0) before closing the tab, and **File** → **Upload .qasm file** to reload it later.

04

Set up Linux for Qiskit

Everything from here happens in a terminal — try Ctrl+Alt+T, or search "Terminal" in your app launcher. Commands below cover Ubuntu/Debian (apt), Fedora (dnf), and Arch (pacman).

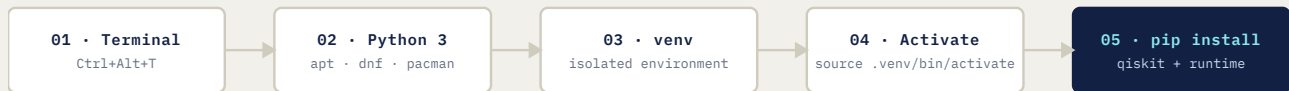


FIG 04 — The five steps between a blank terminal and a working Qiskit install.

01 Install Python 3, pip and venv.

Most distros ship Python 3, but the venv and pip modules are often separate packages — run an update first if your package list is stale:

```
$ sudo apt install python3 python3-venv python3-pip    # Debian/Ubuntu
$ sudo dnf install python3 python3-pip                # Fedora
$ sudo pacman -S python python-pip                   # Arch
```

02 Make a project folder and a virtual environment:

```
$ mkdir -p ~/qiskit-project && cd ~/qiskit-project
$ python3 -m venv .venv
```

IF THIS FAILS

"ensurepip is not available" means the venv package from step 1 isn't installed — `sudo apt install python3-venv` (or your distro's equivalent) and try again.

03 Activate it:

```
$ source .venv/bin/activate
```

Your prompt now starts with `(.venv)`. Run this again whenever you reopen your terminal here.

04 Install Qiskit.

```
$ pip install qiskit qiskit-ibm-runtime matplotlib pylatexenc jupyter
```

Prebuilt wheels cover x86_64 and ARM64 (plus POWER and IBM Z). At writing time this pulls Qiskit 2.5 and qiskit-ibm-runtime 0.49 — Python 3.10+ required.

05 Verify the install:

```
$ python3 -c "import qiskit; print(qiskit.__version__)"
2.5.2
```

05

Save your IBM credentials for Python

Do this once per machine. Qiskit Runtime then finds your account automatically, every time, without you pasting the key again.

```
# run this once, from your activated venv
from qiskit_ibm_runtime import QiskitRuntimeService

QiskitRuntimeService.save_account(
    token="<your 44-character API key>",
    instance="<your instance CRN>",
    set_as_default=True,
)
```



FIG 05 – `save_account()` writes your credentials to disk once; every later script loads them silently.

LOADING IT LATER

Any script can now just call:

```
service = QiskitRuntimeService()
```

KEEP IT PRIVATE

Never paste your API key into code you commit to GitHub or share. If you prefer not to write it to disk at all, set `QISKIT_IBM_TOKEN` and `QISKIT_IBM_INSTANCE` as environment variables instead – Qiskit checks those too.

06

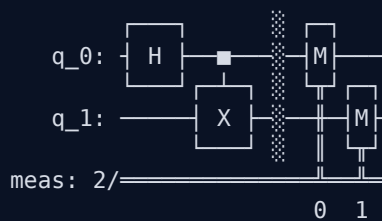
Your first Qiskit program

The same Bell state from Part 03, now in code — build it, look at it, then run it on the simulator that ships inside Qiskit. No queue, no internet, no usage allowance spent.

Build and draw the circuit

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()
print(qc.draw())
```



exact terminal output — this is what `qc.draw()` prints

Run it on the built-in simulator

```
from qiskit.primitives import StatevectorSampler
from qiskit.visualization import plot_histogram

sampler = StatevectorSampler()
job = sampler.run([qc], shots=1024)
counts = job.result()[0].data.meas.get_counts()
print(counts)

plot_histogram(counts).savefig("bell.png")
```

```
$ python3 bell.py
{'00': 508, '11': 516}
```

A real run from this exact script — close to the ideal 50/50 split, as expected for a noise-free simulator with 1024 shots.

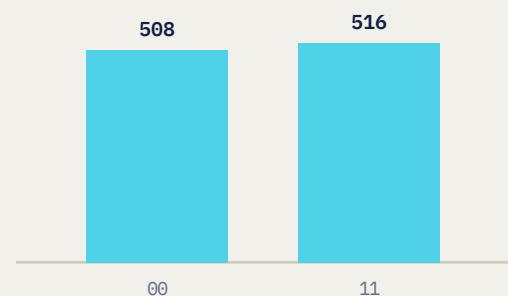


FIG 06 — bell.png, as saved by the script above.

WHY NOT 512 / 512 EXACTLY

A perfect simulator still samples randomly — like flipping a coin 1024 times won't land exactly 512/512. The split tightens toward 50/50 as shot count grows.

07

Run on real IBM hardware

Same circuit, a real chip. The one new step is **transpilation** — rewriting your circuit into the specific gates and qubit connections that chip actually has.

```
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler
from qiskit.transpiler import generate_preset_pass_manager

service = QiskitRuntimeService()
backend = service.least_busy(simulator=False, operational=True)

pm = generate_preset_pass_manager(backend=backend, optimization_level=1)
isa_qc = pm.run(qc) # rewritten for this chip

sampler = Sampler(mode=backend)
job = sampler.run([isa_qc], shots=1024)
print(job.job_id()) # save this - fetch results later

counts = job.result()[0].data.meas.get_counts()
print(counts)
```

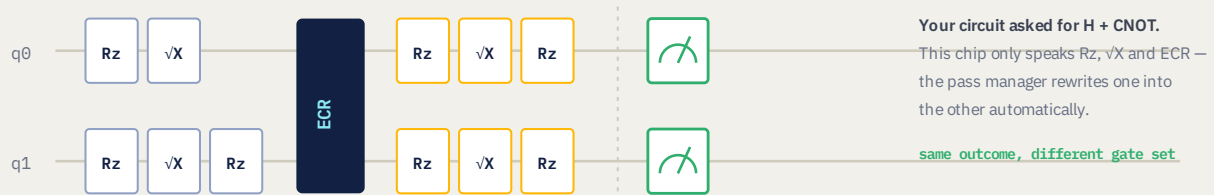


FIG 07 — an actual transpiled gate sequence for this circuit, captured against a simulated backend (real hardware uses its own current basis gates, which can differ chip to chip).

EXPECT SOME NOISE

Real hardware isn't perfect. A typical result looks like {'00': 514, '11': 428, '10': 41, '01': 41} — mostly 00/11, with a handful of stray outcomes from device noise.

NO CHIP HANDY? TEST AGAINST A FAKE ONE

```
from qiskit_ibm_runtime.fake_provider
import FakeSherbrooke
backend = FakeSherbrooke() # same
pm.run() + Sampler code above
```

Simulates a real chip's noise profile locally — no queue, no usage.

08

How the Composer and Qiskit code connect

They're two views of the same thing. Build visually, export to code — or the reverse.

COMPOSER → PYTHON

Open the code editor panel and switch it to the Qiskit view. Copy that straight into a .py file — it's read-only in Composer, but yours to edit once it's in your project.

PYTHON → COMPOSER

Export your circuit as OpenQASM (`qiskit.qasm2.dump(qc, "circuit.qasm")`), then **File** → **Upload .qasm file** in the Composer to see and edit it visually.

The Qiskit "patterns" workflow

Every Qiskit program you write — this one included — follows the same four steps. Recognising them makes any new circuit you read easier to follow.

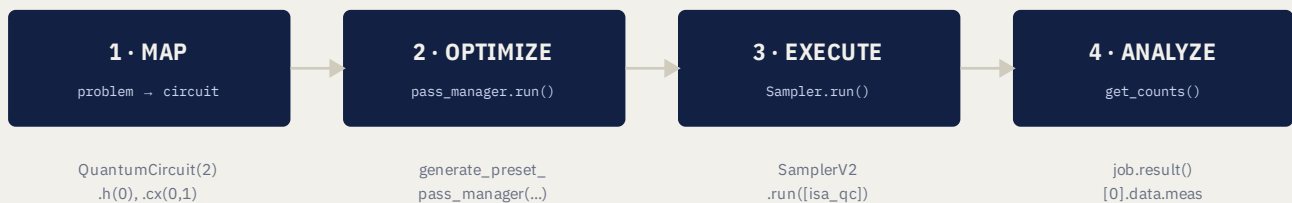
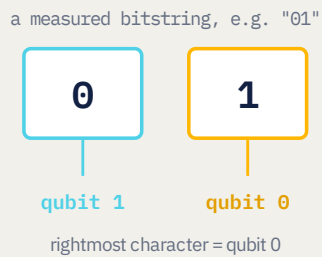


FIG 08 — the four-step pattern behind every Qiskit program, mapped to the exact calls used in Parts 06–07.

09

Reference

Reading a bitstring — least significant qubit is on the right



Qiskit numbers bits so the **least significant bit sits on the right** — the opposite of how the wires are drawn top-to-bottom in a circuit diagram. In '01', qubit 0 measured 1 and qubit 1 measured 0. This trips up almost everyone the first time; it's worth checking deliberately rather than assuming.

Common snags

Symptom	What's going on
Job stuck "queued"	Real QPUs have a waiting line. Develop and debug on <code>StatevectorSampler()</code> or a Fake backend, and only submit to real hardware once the circuit is right.
"No instance" / auth error	Re-check the API key and CRN from Part 01, and confirm you saved them with <code>save_account()</code> in the same environment you're running from.
Sampler vs. Estimator	<code>Sampler</code> returns bitstring counts, the shape used throughout this guide. <code>Estimator</code> instead returns expectation values of an observable — a different question, for a different kind of program.
<code>pip install</code> fails to build	Confirm you're inside the activated venv (prompt starts with <code>(.venv)</code>) and that <code>python3 --version</code> reports 3.10 or newer.
Composer circuit vanished	Sessions aren't saved automatically — File → Save file before closing the tab.

One-page cheat sheet

```
# install
sudo apt install python3-venv python3-pip
python3 -m venv .venv && source
.venv/bin/activate
pip install qiskit qiskit-ibm-runtime
matplotlib

# save credentials, once
QiskitRuntimeService.save_account(
    token=..., instance=...)

```

```
# build → simulate → run
qc = QuantumCircuit(2)
qc.h(0); qc.cx(0,1); qc.measure_all()

sampler = StatevectorSampler()
counts = sampler.run([qc]).result()[0] \
    .data.meas.get_counts()

```

Composer: quantum.cloud.ibm.com/composer · Docs: quantum.cloud.ibm.com/docs · Verified against Qiskit 2.5.2 and qiskit-ibm-runtime 0.49.0, on the Open Plan, September 2026.